



SCHEDA CONCETTO

Cos'è l'Infrastructure as Code?

L'Infrastructure as Code (IaC) è un approccio alla gestione dell'infrastruttura IT che prevede la scrittura e l'esecuzione di codice per definire, distribuire, aggiornare e distruggere l'infrastruttura stessa.

In sostanza, si tratta di utilizzare il linguaggio di programmazione per automatizzare le attività di gestione dell'infrastruttura, in modo da rendere il processo più efficiente e meno soggetto ad errori.



SCHEDA CONCETTO

Quali sono i vantaggi dell'Infrastructure as Code?

L'Infrastructure as Code (IaC) consente di automatizzare la gestione dell'infrastruttura IT attraverso la scrittura e l'esecuzione di codice, offrendo numerosi vantaggi come l'automazione, la riutilizzabilità del codice e la versioning del codice.

Inoltre, IaC rende i cambiamenti idempotenti, consistenti, ripetibili e prevedibili, semplificando il processo di gestione dell'infrastruttura e riducendo il rischio di errori umani.



SCHEMA CONCETTO

Come l'utilizzo dell'Infrastructure as Code semplifica la distribuzione dell'infrastruttura?

L'utilizzo dell'Infrastructure as Code semplifica la distribuzione dell'infrastruttura grazie alla sua automazione e alla sintassi comune utilizzata tra i diversi provider di infrastruttura (ad esempio, AWS, Azure, GCP, ecc.).

Questo consente di risparmiare tempo e di standardizzare i processi di lavoro, rendendo più facile la configurazione e la gestione dell'infrastruttura.



SCHEDA CONCETTO

Cosa significa "Ideomponent" in termini di IaC?

La caratteristica "Ideomponent" fornita dagli strumenti IaC garantisce che, anche se lo stesso codice viene applicato più volte, il risultato rimane lo stesso.



SCHEMA CONCETTO

Cosa sono le attività del "Giorno 0" e del "Giorno 1"?

IaC può essere applicato durante l'intero ciclo di vita dell'infrastruttura, sia durante la costruzione iniziale che nel corso della vita dell'infrastruttura stessa.

Comunemente, queste attività sono denominate attività del "Giorno 0" e del "Giorno 1".

Il codice del "Giorno 0" fornisce e configura l'infrastruttura iniziale.

Il "Giorno 1" si riferisce alle configurazioni dell'OS e dell'applicazione che si applicano dopo aver costruito inizialmente l'infrastruttura.



SCHEDA CONCETTO

Quali sono i casi d'uso di Terraform?

- Applicazioni a più livelli
- Cluster self-service
- Demo software
- Ambienti usa e getta
- Networking definito dal software
- Pianificatori di risorse
- Distribuzione multi-cloud



SCHEDA CONCETTO

Quali sono i vantaggi di Terraform?

Piattaforma agnostica: Terraform è in grado di gestire la gestione dell'infrastruttura su molte piattaforme cloud e on-premise.

Gestione dello stato: Terraform tiene traccia dello stato dell'infrastruttura in modo che gli operatori possano visualizzare facilmente le differenze tra l'infrastruttura esistente e quella pianificata.

Fiducia degli operatori: Terraform permette di gestire l'infrastruttura in modo più sicuro e affidabile, consentendo agli operatori di effettuare modifiche con maggiore sicurezza e prevedibilità.



SCHEDA CONCETTO

Dove descrivi tutti i componenti o l'intero data center in modo che Terraform li possa configurare?

Tutti i componenti dell'infrastruttura e il data center possono essere descritti nei file di configurazione di Terraform, che hanno l'estensione ".tf".

In questi file, è possibile definire i provider, i dati, le risorse e i moduli che descrivono l'intera infrastruttura. Terraform utilizzerà poi questi file di configurazione per configurare e gestire l'infrastruttura specificata.



SCHEDA CONCETTO

Come può Terraform costruire l'infrastruttura in modo così efficiente?

Terraform costruisce un grafo di tutte le risorse e parallelizza la creazione e la modifica di tutte le risorse che non dipendono l'una dall'altra.

Grazie a ciò, Terraform è in grado di costruire l'infrastruttura in modo altamente efficiente, e gli operatori ottengono una panoramica delle dipendenze presenti nella loro infrastruttura.



SCHEDA CONCETTO

Cos'è la distribuzione multi-cloud?

La distribuzione multi-cloud consiste nella configurazione dell'infrastruttura su più provider cloud allo scopo di aumentare la tolleranza ai guasti delle applicazioni.

In questo modo, se un cloud provider dovesse avere un'interruzione del servizio o un guasto, l'applicazione può essere comunque eseguita su un altro provider senza interruzioni significative.



SCHEMA CONCETTO

In che modo la distribuzione multi-cloud è utile?

L'utilizzo di una singola regione o di un singolo provider cloud limita la tolleranza ai guasti alla disponibilità di quel provider.

L'utilizzo di una distribuzione multi-cloud consente una ripresa più rapida e senza interruzioni in caso di perdita di una regione o di un intero provider.

In questo modo, si riducono i rischi di perdita di dati, interruzioni del servizio e rallentamenti delle applicazioni.



SCHEDA CONCETTO

Qual è l'utilizzo di Terraform che è agnostico rispetto alla piattaforma cloud?

Ciò semplifica la gestione e l'orchestrazione, aiutando gli operatori a costruire infrastrutture multi-cloud su larga scala in modo più facile e standardizzato.

Con Terraform, gli operatori possono gestire l'infrastruttura su più piattaforme cloud e on-premise in modo coerente, riducendo i costi, migliorando la scalabilità e aumentando la flessibilità della loro infrastruttura.



SCHEDA CONCETTO

Cos'è lo stato di Terraform?

Ogni volta che viene eseguito Terraform, registra le informazioni sull'infrastruttura che ha creato in un file di stato di Terraform. Per impostazione predefinita, quando si esegue Terraform nella cartella `/some/folder`, Terraform crea il file `/some/folder/terraform.tfstate`.

Questo file contiene un formato JSON personalizzato che registra una mappatura delle risorse Terraform nei file di configurazione alla rappresentazione di quelle risorse nel mondo reale.

Lo stato di Terraform è fondamentale per la gestione dell'infrastruttura e consente di effettuare il deployment, la modifica e la rimozione delle risorse in modo coerente e prevedibile.



SCHEDA CONCETTO

Dove si inseriscono le configurazioni di Terraform per configurare alcuni comportamenti di Terraform stesso?

Le configurazioni di Terraform che servono a configurare alcuni comportamenti di Terraform stesso, ad esempio richiedere una versione minima di Terraform per applicare la configurazione, vengono inserite all'interno di un blocco di configurazione speciale denominato "terraform".

Il blocco "terraform" viene inserito all'inizio del file di configurazione di Terraform e include le opzioni di configurazione di Terraform stessa.



SCHEDA CONCETTO

Cosa sono i Provider?

Un provider è un plugin che Terraform utilizza per tradurre le interazioni con l'API del servizio.

Un provider è responsabile di comprendere le interazioni con l'API e di esporre le risorse.

Poiché Terraform può interagire con qualsiasi API, è possibile rappresentare quasi ogni tipo di infrastruttura come risorsa in Terraform. I provider di Terraform includono servizi come AWS, Azure, Google Cloud Platform, VMware, e molti altri.



SCHEDA CONCETTO

Come si configura un Provider?

Per configurare un provider in Terraform, si utilizza un blocco di configurazione che specifica il nome del provider da configurare, ad esempio "google", e i relativi argomenti di configurazione, ad esempio il progetto e la regione.



SCHEMA CONCETTO

Quali sono i meta-argomenti definiti da Terraform e disponibili per tutti i blocchi di provider?

Alcuni degli argomenti speciali definiti da Terraform e disponibili per tutti i blocchi di provider includono:

- Version: Limita le versioni del provider consentite.
- Alias: Consente di utilizzare lo stesso provider con diverse configurazioni per risorse diverse.



SCHEDA CONCETTO

Cos'è l'inizializzazione del Provider e perché ne abbiamo bisogno?

Ogni volta che viene aggiunto un nuovo provider alla configurazione, sia esplicitamente tramite un blocco provider che aggiungendo una risorsa da quel provider, Terraform deve inizializzare il provider prima che possa essere utilizzato.

L'inizializzazione scarica e installa il plugin del provider in modo che possa essere eseguito in seguito.

L'inizializzazione del provider è necessaria per configurare correttamente il provider e per garantire che tutte le risorse correlate siano gestite correttamente.



SCHEDA CONCETTO

Come si inizializza un Provider in Terraform?

L'inizializzazione del provider è una delle azioni di *terraform init*.

L'esecuzione di questo comando scaricherà e inizierà tutti i provider che non sono già stati inizializzati.

Inoltre, è possibile forzare l'inizializzazione di un singolo provider tramite il comando *terraform init -upgrade*, che installerà l'ultima versione del provider specificato nella configurazione.



SCHEDA CONCETTO

Come si configurano più istanze di Provider in Terraform?

È possibile configurare più istanze di uno stesso provider utilizzando l'opzione "alias".

In questo modo, è possibile definire più configurazioni per lo stesso provider e selezionare quale utilizzare per risorsa o modulo.



SCHEDA CONCETTO

Cos'è il file di configurazione della CLI di Terraform?

Il file di configurazione della CLI di Terraform (CLI configuration file) permette di configurare le impostazioni dell'interfaccia della riga di comando di Terraform a livello utente, che si applicano a tutte le directory di lavoro di Terraform.

Questo file può essere denominato `.terraformrc` o `terraform.rc`.

Alcune delle opzioni di configurazione possibili includono la configurazione del colore dell'output della riga di comando, l'impostazione del timeout delle operazioni di Terraform, la configurazione di un mirror per la configurazione dei provider, e così via.



SCHEMA CONCETTO

Qual è il comando per inizializzare la directory di lavoro di Terraform?

Il comando per inizializzare la directory di lavoro di Terraform è `terraform init`.

Questo comando scarica e installa i plugin dei provider specificati nella configurazione, inizializza lo stato di Terraform e configura altri elementi necessari per l'esecuzione delle operazioni di Terraform.

È una buona pratica eseguire il comando `terraform init` ogni volta che si cambia directory di lavoro o si modifica la configurazione di Terraform.



SCHEMA CONCETTO

Se diversi team stanno lavorando sulla stessa configurazione, come si fa per rendere i file coerenti nella formattazione?

Per garantire la coerenza nella formattazione dei file di configurazione di Terraform, si può utilizzare il comando *terraform fmt*.

Questo comando formatta automaticamente i file di configurazione nella directory corrente per garantire una leggibilità uniforme e la coerenza della formattazione.

In questo modo, si evita la confusione tra team che utilizzano diversi stili di formattazione, mantenendo la leggibilità dei file di configurazione per tutti i membri del team.



SCHEDA CONCETTO

Se diversi team stanno lavorando sulla stessa configurazione, come si fa per garantire che i file siano sintatticamente validi e coerenti internamente?

Per garantire che i file di configurazione di Terraform siano sintatticamente validi e coerenti internamente, si può utilizzare il comando *terraform validate*.

Questo comando verifica la correttezza sintattica della configurazione e segnala eventuali errori presenti nei moduli, nei nomi degli attributi e nei tipi di valore.

In questo modo, si evita di introdurre errori nel codice e si assicura che la configurazione sia coerente e valida.

Se la configurazione è valida, Terraform restituirà un messaggio di successo.



SCHEDA CONCETTO

Qual è il comando per creare l'infrastruttura con Terraform?

Il comando per creare l'infrastruttura con Terraform è *terraform apply*. Questo comando applica la configurazione specificata nella directory corrente e crea o modifica le risorse necessarie per implementare la configurazione desiderata.

Prima di eseguire il comando *terraform apply*, è consigliabile eseguire il comando *terraform plan* per visualizzare le modifiche proposte e assicurarsi che la configurazione sia coerente con le aspettative.



SCHEDA CONCETTO

Qual è il comando per visualizzare il piano di esecuzione delle modifiche in Terraform senza applicarle?

Il comando per visualizzare il piano di esecuzione delle modifiche in Terraform senza applicarle è *terraform plan*.

Questo comando mostra una panoramica delle modifiche che verranno apportate all'infrastruttura in base alla configurazione corrente.

In questo modo è possibile visualizzare le modifiche proposte e valutare gli eventuali rischi o impatti prima di applicare effettivamente le modifiche tramite il comando *terraform apply*.



SCHEDA CONCETTO

Come si ispeziona lo stato corrente dell'infrastruttura applicata in Terraform?

Per ispezionare lo stato corrente dell'infrastruttura applicata in Terraform, si può utilizzare il comando *terraform show*. Questo comando visualizza il contenuto del file di stato di Terraform (*terraform.tfstate*) in un formato leggibile dall'utente, mostrando gli ID e le proprietà delle risorse create da Terraform. In questo modo è possibile controllare lo stato corrente dell'infrastruttura e assicurarsi che sia coerente con la configurazione desiderata.



SCHEDA CONCETTO

Cos'è l'architettura basata su plug-in?

L'architettura basata su plug-in è un modello di sviluppo software in cui le funzionalità aggiuntive sono definite come plug-in per la piattaforma o l'applicazione principale.

Questo modello fornisce estensibilità, flessibilità e isolamento, in quanto consente di aggiungere funzionalità a un'applicazione esistente senza doverla modificare direttamente, ma semplicemente utilizzando i plug-in corrispondenti.

Questa architettura è utilizzata in molte applicazioni e framework, inclusi i sistemi di gestione dei contenuti, i browser web, i sistemi operativi, le applicazioni di produttività e molte altre.



SCHEDA CONCETTO

Cosa sono i Provisioner in Terraform?

I provisioner in Terraform sono una funzionalità che consente di eseguire attività di configurazione o setup iniziale sulle risorse di infrastruttura gestite da Terraform.

Ad esempio, se è necessario eseguire l'installazione di software aggiuntivi, eseguire script shell, caricare file o avviare altri strumenti di gestione della configurazione, i provisioner consentono di automatizzare queste attività in modo da semplificare la configurazione dell'infrastruttura.

I provisioner in Terraform includono diverse opzioni, come l'uso di script shell, l'installazione di software o l'esecuzione di comandi personalizzati.



SCHEMA CONCETTO

Quali sono i tipi di provisioner in Terraform?

I tipi di provisioner in Terraform includono:

- local-exec: questo tipo di provisioner esegue script o comandi sul computer locale in cui viene eseguito Terraform, ad esempio per eseguire script shell o comandi di sistema operativo.
- remote-exec: questo tipo di provisioner esegue script o comandi sulla risorsa remota creata da Terraform, ad esempio per installare pacchetti software o eseguire script shell sulle istanze di una macchina virtuale.



SCHEDA CONCETTO

Cos'è un provisioner local-exec e quando viene utilizzato?

Il provisioner local-exec esegue un comando localmente sulla macchina in cui viene eseguito Terraform.

Viene utilizzato quando è necessario eseguire un'azione sulla macchina locale, senza la necessità di accedere a un URL o a una risorsa esterna. Ad esempio, il provisioner local-exec può essere utilizzato per eseguire uno script shell o un comando di sistema operativo sulla macchina locale dopo che Terraform ha creato o modificato una risorsa di infrastruttura, come ad esempio la creazione di un'istanza di macchina virtuale.



SCHEDA CONCETTO

Cos'è un provisioner remote-exec e quando viene utilizzato?

Il provisioner remote-exec viene utilizzato per eseguire uno script su una risorsa remota dopo che è stata creata da Terraform.

Questo tipo di provisioner viene spesso utilizzato per avviare uno strumento di gestione della configurazione, come Ansible o Puppet, sulla risorsa appena creata per configurarla ulteriormente.

Inoltre, il provisioner remote-exec può essere utilizzato per eseguire script di bootstrap in un cluster, configurare le connessioni di rete o installare il software necessario.

In sostanza, il provisioner remote-exec consente di eseguire azioni su una risorsa appena creata, ad esempio un'istanza di macchina virtuale o un server, per configurarla o prepararla per l'uso.



SCHEMA CONCETTO

Quando Terraform segna le risorse come "tainted"?

Se una risorsa viene creata con successo ma fallisce durante il provisioning, Terraform restituisce un errore e segna la risorsa come "tainted".

Una risorsa contrassegnata come "tainted" è stata creata fisicamente, ma non può essere considerata sicura da utilizzare poiché il provisioning è fallito.

In pratica, significa che è necessario eseguire il provisioning manualmente o eseguire nuovamente il provisioning della risorsa utilizzando i provisioner appropriati per garantire che sia configurata correttamente.



SCHEDA CONCETTO

Perché Terraform non esegue automaticamente il rollback e la distruzione della risorsa durante l'apply quando si verifica un errore?

Terraform non esegue automaticamente il rollback e la distruzione della risorsa durante l'apply quando si verifica un errore, poiché ciò andrebbe contro il piano di esecuzione: il piano di esecuzione avrebbe indicato che una risorsa sarebbe stata creata, ma non avrebbe mai detto che sarebbe stata eliminata.

Tuttavia, se si crea un piano di esecuzione con una risorsa contrassegnata come "tainted", il piano indicherà chiaramente che la risorsa verrà distrutta perché è tainted.

In pratica, questo significa che l'utente deve gestire manualmente la risorsa tainted e provvedere alla sua configurazione corretta o, eventualmente, alla sua eliminazione.



SCHEDA CONCETTO

Come si applicano i vincoli alle versioni dei provider in Terraform?

Per applicare i vincoli alle versioni dei provider in Terraform, è possibile utilizzare la direttiva *required_providers* all'interno del blocco terraform del file di configurazione.

La direttiva *required_providers* è una mappa che specifica un vincolo di versione per ogni provider richiesto dalla configurazione.



SCHEDA CONCETTO

Come si configura la versione richiesta del CLI di Terraform da utilizzare con la propria configurazione?

Per configurare la versione richiesta del CLI di Terraform da utilizzare con la propria configurazione, è possibile utilizzare la direttiva *required_version* all'interno del blocco terraform del file di configurazione.

La direttiva *required_version* consente di vincolare le versioni del CLI di Terraform che possono essere utilizzate con la propria configurazione. Se la versione in esecuzione di Terraform non corrisponde ai vincoli specificati, Terraform produrrà un errore ed uscirà senza eseguire ulteriori azioni.



SCHEMA CONCETTO

Cosa fa il comando `fmt` in Terraform?

Il comando *terraform fmt* viene utilizzato per riscrivere i file di configurazione di Terraform in un formato e uno stile canonici. Questo comando applica un sottoinsieme delle convenzioni di stile del linguaggio Terraform, insieme ad altri piccoli aggiustamenti per migliorare la leggibilità. In pratica, il comando `terraform fmt` applica una formattazione uniforme ai file di configurazione di Terraform, migliorando la leggibilità e la manutenibilità della configurazione.



SCHEDA CONCETTO

Qual è l'approccio consigliato dopo l'aggiornamento di Terraform?

Il formato canonico può cambiare leggermente tra le versioni di Terraform, quindi dopo l'aggiornamento di Terraform si consiglia di eseguire in modo proattivo il comando *terraform fmt* sui moduli insieme a eventuali altre modifiche che si stanno apportando per adottare la nuova versione.

In questo modo si garantirà la corretta formattazione dei file di configurazione e si eviteranno problemi di compatibilità e di leggibilità della configurazione.



SCHEDA CONCETTO

Se stiamo formattando i file di configurazione, quale flag deve essere utilizzato per visualizzare le differenze?

Il flag da utilizzare per visualizzare le differenze tra i file di configurazione prima e dopo la formattazione è *-diff*.

Quindi, per eseguire la formattazione e visualizzare le differenze, è necessario eseguire il comando *terraform fmt* con il flag *-diff*.



SCHEMA CONCETTO

Qual è il nome predefinito dello spazio di lavoro in Terraform?

Il nome predefinito dello spazio di lavoro in Terraform è "*default*".
Quando si utilizza Terraform, si opera sempre in uno spazio di lavoro, anche se non viene specificato alcun nome di spazio di lavoro durante l'utilizzo dei comandi Terraform.
In tal caso, Terraform utilizzerà automaticamente lo spazio di lavoro predefinito.



SCHEDA CONCETTO

Cosa sono gli spazi di lavoro (workspaces) in Terraform?

Ogni configurazione Terraform ha un backend associato che definisce come vengono eseguite le operazioni e dove vengono memorizzati i dati persistenti come lo stato di Terraform.

I dati persistenti memorizzati nel backend appartengono a uno spazio di lavoro (workspace).

Inizialmente, il backend ha un solo spazio di lavoro, chiamato "default", e quindi c'è solo uno stato Terraform associato a quella configurazione. Alcuni backend supportano più spazi di lavoro denominati, consentendo a più stati di essere associati a una singola configurazione.



SCHEMA CONCETTO

Qual è il comando per elencare gli spazi di lavoro (workspaces)?

Il comando per elencare gli spazi di lavoro (workspaces) è *terraform workspace list*.



SCHEMA CONCETTO

Qual è il comando per mostrare lo spazio di lavoro (workspace) corrente?

Il comando per mostrare lo spazio di lavoro (workspace) corrente è *terraform workspace show*.



SCHEMA CONCETTO

Cos'è il comando *state*?

Il comando *terraform state* viene utilizzato per la gestione avanzata dello stato.

Quando si utilizza Terraform in modo più avanzato, ci sono casi in cui potrebbe essere necessario modificare lo stato di Terraform.

Invece di modificare lo stato direttamente, i comandi di *terraform state* possono essere utilizzati in molti casi.



SCHEMA CONCETTO

Quando si utilizzano gli spazi di lavoro (workspaces), dove salva Terraform il file di stato per lo stato locale?

Terraform salva gli stati degli spazi di lavoro in una directory chiamata *terraform.tfstate.d*.

Questo vale per lo stato locale, cioè quando non si utilizza un backend remoto per memorizzare lo stato.



SCHEDA CONCETTO

Cosa è un modulo Terraform?

Un modulo Terraform è un insieme di file di configurazione Terraform in una singola directory.

Anche una configurazione semplice composta da una singola directory con uno o più file .tf è un modulo.

Tipicamente, tuttavia, un modulo Terraform ha una struttura più complessa, contenente molteplici risorse e fonti di dati, insieme a variabili di input e output che possono essere utilizzate per personalizzare il comportamento del modulo quando viene utilizzato all'interno di una configurazione più ampia.

L'utilizzo dei moduli Terraform consente di organizzare in modo efficace la configurazione di infrastrutture complesse e riutilizzare la configurazione in più ambienti.



SCHEMA CONCETTO

Quante modalità ci sono per caricare i moduli?

I moduli possono essere caricati in due modi: da una fonte locale o da una fonte remota.

Terraform supporta una varietà di fonti remote, tra cui il registro Terraform, la maggior parte dei sistemi di controllo versione, URL HTTP e i registri privati di moduli di Terraform Cloud o Terraform Enterprise.



SCHEDA CONCETTO

Quando si utilizzano moduli locali, non è necessario eseguire il comando `init` o `get` ogni volta che viene apportata una modifica al modulo locale. Perché?

Quando si installa un modulo locale, Terraform fa riferimento direttamente alla directory di origine.

A causa di ciò, Terraform noterà automaticamente le modifiche ai moduli locali senza dover eseguire nuovamente il comando *terraform init* o *terraform get*.



SCHEMA CONCETTO

Quando si utilizzano moduli remoti, cosa bisogna fare in caso di una modifica al modulo?

Quando si installa un modulo remoto, Terraform lo scarica nella directory `.terraform` nella directory radice della propria configurazione.

In caso di una modifica al modulo remoto, è necessario eseguire il comando `terraform init` per aggiornare la configurazione con le ultime modifiche del modulo.



SCHEMA CONCETTO

Qual è il flusso di lavoro di base di Terraform?

Il flusso di lavoro di base di Terraform prevede tre fasi:

- WRITE - Scrivere l'infrastruttura come codice.
- PLAN - Anteprima delle modifiche prima dell'applicazione.
- APPLY - Provisioning dell'infrastruttura riproducibile.



SCHEDA CONCETTO

Cos'è il comando *init*?

Il comando *terraform init* viene utilizzato per inizializzare una directory di lavoro contenente i file di configurazione Terraform.

Questo è il primo comando che deve essere eseguito dopo aver scritto una nuova configurazione Terraform o dopo aver clonato una configurazione esistente da un sistema di controllo di versione.

È sicuro eseguire questo comando più volte.



SCHEDA CONCETTO

Se durante l'inizializzazione con il comando `terraform init` si vuole saltare l'installazione dei plugin, cosa bisogna fare?

È possibile utilizzare il flag `-get-plugins=false` con il comando `terraform init` per saltare l'installazione dei plugin.

In questo modo, Terraform utilizzerà i plugin già installati nella directory dei plugin dell'utente e quelli già installati per la directory di lavoro corrente.

Se i plugin installati non sono sufficienti per la configurazione, l'inizializzazione fallirà.



SCHEDA CONCETTO

Cosa fa il comando `terraform validate`?

Il comando *terraform validate* valida i file di configurazione in una directory, facendo riferimento solo alla configurazione e non accedendo a servizi remoti come lo stato remoto, le API dei provider, ecc.

Il comando *Validate* esegue controlli per verificare se una configurazione è sintatticamente valida e internamente coerente, indipendentemente dalle variabili fornite o dallo stato esistente.

In sostanza, questo comando controlla se la configurazione Terraform è scritta correttamente e può essere utilizzata per creare o modificare l'infrastruttura.



SCHEDA CONCETTO

Cosa fa il comando `terraform plan`?

Il comando *terraform plan* viene utilizzato per creare un piano di esecuzione.

Terraform esegue un aggiornamento (refresh), a meno che non sia esplicitamente disabilitato, e poi determina quali azioni sono necessarie per raggiungere lo stato desiderato specificato nei file di configurazione. In sostanza, il comando *plan* mostra un'anteprima delle modifiche che Terraform effettuerà all'infrastruttura esistente per farla corrispondere alla configurazione desiderata, senza apportare effettivamente le modifiche.

Il risultato di questo comando è un report dettagliato delle modifiche previste, tra cui le creazioni, le modifiche e le cancellazioni di risorse.



SCHEDA CONCETTO

Cosa fa il comando `terraform apply`?

Il comando *terraform apply* viene utilizzato per applicare le modifiche necessarie per raggiungere lo stato desiderato della configurazione o l'insieme predefinito di azioni generate da un piano di esecuzione di terraform plan.

In sostanza, questo comando effettua le modifiche previste sulle risorse dell'infrastruttura, come mostrato dal piano di esecuzione creato dal comando *terraform plan*.

Una volta avviato, Terraform applicherà le modifiche richieste all'infrastruttura e, se tutto va bene, creerà o modificherà le risorse in base alla configurazione specificata.

Il comando *apply* richiederà una conferma prima di applicare le modifiche e può essere annullato in qualsiasi momento prima di completare l'applicazione.



SCHEDA CONCETTO

Cosa fa il comando `terraform destroy`?

Il comando *terraform destroy* viene utilizzato per distruggere l'infrastruttura gestita da Terraform. In sostanza, questo comando rimuove tutte le risorse create dall'applicazione della configurazione Terraform.

Una volta avviato, Terraform chiederà una conferma prima di distruggere le risorse e, se confermato, distruggerà tutte le risorse specificate nella configurazione, ad eccezione delle risorse che sono protette da uno stato remoto o da un blocco di lock.

È importante usare questo comando con cautela, poiché le risorse distrutte non possono essere recuperate.



SCHEDA CONCETTO

Come si può visualizzare un'anteprima del comportamento del comando `terraform destroy`?

Per visualizzare un'anteprima del comportamento del comando *terraform destroy*, è possibile utilizzare il comando `terraform plan` con l'opzione *-destroy*.

In questo modo, verrà creato un piano di esecuzione che mostrerà l'elenco delle risorse che verranno distrutte e tutti i relativi dettagli, senza effettuare effettivamente la distruzione delle risorse stesse. Questo comando consente di valutare l'impatto della distruzione delle risorse e di apportare eventuali correzioni necessarie prima di effettuare la distruzione effettiva.



SCHEMA CONCETTO

Cosa sono le dipendenze implicite?

Studiando gli attributi delle risorse utilizzati nelle espressioni di interpolazione, Terraform può dedurre automaticamente quando una risorsa dipende da un'altra.

Terraform utilizza queste informazioni di dipendenza per determinare l'ordine corretto in cui creare le diverse risorse.

Le dipendenze implicite tramite le espressioni di interpolazione sono il modo principale per informare Terraform su queste relazioni e dovrebbero essere utilizzate ogni volta che possibile.



SCHEDA CONCETTO

Cosa sono le dipendenze esplicite?

A volte ci sono dipendenze tra le risorse che non sono visibili per Terraform. L'argomento *depends_on* è accettato da qualsiasi risorsa e accetta un elenco di risorse per creare dipendenze esplicite. Questo permette di definire manualmente la dipendenza tra risorse che non possono essere dedotte in modo implicito dalle espressioni di interpolazione.



SCHEDA CONCETTO

Come si salva il piano di esecuzione?

È possibile salvare il piano di esecuzione utilizzando il comando *terraform plan* con l'opzione *-out*, specificando il nome del file di output. Ad esempio, il comando "*terraform plan -out=tfplan*" salva il piano di esecuzione nel file chiamato "*tfplan*".

Successivamente, è possibile utilizzare questo file con il comando *terraform apply* specificando il nome del file come argomento, ad esempio "*terraform apply tfplan*". In questo modo, Terraform applicherà le modifiche esattamente come previsto dal piano di esecuzione salvato nel file.



SCHEMA CONCETTO

Quale flag dovresti utilizzare con il comando `terraform plan` per ottenere dettagli sugli exit code?

Per ottenere maggiori dettagli sugli exit code generati dal comando *terraform plan*, è possibile utilizzare l'opzione *-detailed-exitcode*. Quando viene fornita, questa opzione cambia i codici di uscita e il loro significato per fornire informazioni più dettagliate su cosa contiene il piano di esecuzione risultante:

- 0 = Riuscito con diff vuoto (nessuna modifica)
- 1 = Errore
- 2 = Riuscito con diff non vuoto (modifiche presenti)



SCHEDA CONCETTO

Come si possono selezionare solo risorse specifiche quando si esegue un piano di esecuzione con Terraform?

Per selezionare solo risorse specifiche durante l'esecuzione del comando `terraform plan`, è possibile utilizzare l'opzione `-target=resource`, dove "resource" è l'indirizzo della risorsa che si desidera selezionare. Questa opzione consente di limitare il piano di esecuzione solo alle risorse selezionate, invece di generare un piano per l'intera configurazione.



SCHEMA CONCETTO

Come si aggiorna lo stato prima di verificare le differenze quando si esegue un piano di esecuzione con Terraform?

Per aggiornare lo stato di Terraform prima di verificare le differenze durante l'esecuzione del comando *terraform plan*, è possibile utilizzare l'opzione *-refresh=true*.

Questa opzione consente di eseguire un aggiornamento delle risorse prima della verifica delle differenze, in modo da garantire che il piano di esecuzione rifletta lo stato attuale delle risorse nell'infrastruttura.



SCHEDA CONCETTO

Cosa sono i Backends?

In Terraform, un "backend" determina come viene caricato lo stato e come viene eseguita un'operazione come "apply".

Questa astrazione consente lo storage dello stato in una posizione non locale e l'esecuzione remota delle operazioni di Terraform.

Per impostazione predefinita, Terraform utilizza il backend "local", che corrisponde al comportamento normale di Terraform.

Tuttavia, Terraform supporta anche una vasta gamma di backend remoti, tra cui Amazon S3, Microsoft Azure Storage, Consul, etcd e molti altri. L'utilizzo di un backend remoto consente di gestire lo stato di Terraform in modo centralizzato e di consentire la collaborazione tra team.



SCHEDA CONCETTO

Cos'è il backend locale?

Il backend locale memorizza lo stato su filesystem locale, blocca lo stato utilizzando le API di sistema e esegue le operazioni localmente.

Quando si utilizza il backend locale, lo stato di Terraform viene memorizzato in un file di stato sul filesystem locale del computer in uso. Questo tipo di backend non richiede di configurare una connessione a un servizio remoto e pertanto è facile da usare, ma presenta alcune limitazioni rispetto ai backend remoti.

Ad esempio, il backend locale non offre un meccanismo di blocco distribuito, il che significa che l'utilizzo simultaneo di Terraform da parte di più utenti o macchine può causare problemi di conflitto.



SCHEDA CONCETTO

Qual è il percorso predefinito per il backend locale?

Il percorso predefinito per il backend locale è "terraform.tfstate", che si trova per impostazione predefinita nella directory del modulo principale della configurazione Terraform.

Questo significa che se non viene specificato alcun percorso personalizzato, Terraform utilizzerà il file di stato "terraform.tfstate" per memorizzare lo stato della configurazione.

Il percorso del file di stato può essere personalizzato utilizzando il parametro "path" nel blocco di configurazione del backend locale nel file di configurazione di Terraform.



SCHEDA CONCETTO

Cos'è il locking dello stato?

Il locking dello stato in Terraform è un meccanismo che garantisce l'integrità dello stato durante le operazioni di scrittura, impedendo che più utenti o processi accedano contemporaneamente allo stesso stato e possano causare conflitti o corruzioni del file di stato.

Il locking dello stato viene gestito dal backend di Terraform e avviene automaticamente su tutte le operazioni che possono scrivere sullo stato, come "apply" o "destroy".

Se il backend supporta il locking dello stato, Terraform acquisirà il lock prima di eseguire l'operazione e lo rilascerà solo alla fine dell'operazione.



SCHEDA CONCETTO

È possibile disattivare il locking dello stato in Terraform?

Sì, è possibile disattivare il locking dello stato in Terraform per la maggior parte dei comandi, utilizzando l'opzione "-lock=false".

Tuttavia, questa opzione non è consigliata in quanto può causare problemi di integrità dello stato e interferire con la gestione delle dipendenze tra risorse.

Il locking dello stato è un'importante funzionalità di sicurezza di Terraform e dovrebbe essere mantenuto attivo nella maggior parte dei casi.

Ci possono essere situazioni in cui la disattivazione del locking dello stato può essere giustificata, ad esempio in ambienti di sviluppo o test in cui non è necessario garantire la stessa affidabilità dell'ambiente di produzione



SCHEMA CONCETTO

Come si assegnano valori predefiniti alle variabili in Terraform?

In Terraform, è possibile assegnare valori predefiniti alle variabili utilizzando il parametro "default" nel blocco di dichiarazione della variabile.



SCHEDA CONCETTO

Cosa sono le variabili di output (output variables) in Terraform?

In Terraform, le variabili di output (output variables) sono utilizzate per organizzare i dati in modo da renderli facilmente consultabili e visualizzabili dall'utente.

Le variabili di output sono un modo per indicare a Terraform quali dati sono importanti.

Questi dati vengono visualizzati quando viene eseguito il comando *"terraform apply"* e possono essere consultati utilizzando il comando *"terraform output"*.



SCHEDA CONCETTO

Come si visualizzano e si consultano le variabili di output in Terraform?

Per visualizzare le variabili di output in Terraform, è possibile utilizzare il comando "*terraform apply*" per applicare la configurazione e generare l'output.

L'output viene visualizzato direttamente sulla console di Terraform e può essere consultato per verificare i valori delle variabili di output.



SCHEDA CONCETTO

Cosa sono i dynamic blocks in Terraform?

I dynamic blocks sono una funzionalità di Terraform che consente di creare blocchi annidati ripetibili in modo dinamico all'interno di altre risorse, utilizzando una sintassi speciale.

Questa funzionalità è utilizzata principalmente per gestire blocchi ripetibili che non accettano espressioni.



SCHEDA CONCETTO

Cosa sono le funzioni integrate (Built-in Functions) in Terraform?

Le funzioni integrate (Built-in Functions) sono una serie di funzioni predefinite che fanno parte del linguaggio di Terraform. Queste funzioni possono essere utilizzate all'interno di espressioni per trasformare e combinare valori.

Le funzioni integrate offrono una vasta gamma di funzionalità, tra cui la manipolazione di stringhe, la gestione di liste e mappe, la conversione di valori tra diversi tipi di dati e molto altro ancora.



SCHEDA CONCETTO

Cos'è Sentinel in Terraform?

Sentinel è un framework integrato di policy-as-code, utilizzato in combinazione con i prodotti Enterprise di HashiCorp come Terraform Enterprise.

Sentinel consente di definire e implementare politiche di sicurezza personalizzate in modo centralizzato, garantendo che tutte le risorse gestite da Terraform rispettino tali politiche.

Con Sentinel, è possibile definire politiche di sicurezza basate su logica e regole specifiche del settore, come ad esempio le politiche di conformità PCI, HIPAA e SOC 2.

Le politiche vengono definite tramite un linguaggio di programmazione integrato e possono accedere a informazioni provenienti da diverse fonti esterne.



SCHEDA CONCETTO

Cos'è il Private Module Registry in Terraform?

Il Private Module Registry in Terraform Cloud è un registro privato di moduli Terraform utilizzato per condividere moduli tra i membri dell'organizzazione.

Questo registro include funzionalità di versionamento dei moduli, una lista filtrabile e cercabile dei moduli disponibili e un configuratore di moduli per creare nuovi workspace in modo più rapido.